

A Study on Neural Network-Based Target Tracking Algorithms for Intelligent Robot

Haonan Zhu*

Wanjiang University of Technology

Ma'anshan, China

*Corresponding author: 724496282@qq.com

Yehong Wu

Wanjiang University of Technology

Ma'anshan, China

1204352717@qq.com

Abstract—Addressing issues such as target motion uncertainty, environmental disturbances, and the difficulty of modeling nonlinear systems when intelligent robots perform target tracking in complex dynamic environments. This paper proposes a neural network-based target tracking algorithm. The algorithm employs a radial basis function neural network to perform online learning and approximation of the target motion model, and combines this with an adaptive sliding mode control strategy to enable the robot to accurately track moving targets. Additionally, an improved weight update law is designed to ensure the convergence and stability of the neural network learning process. MATLAB simulation results demonstrate that the proposed algorithm can effectively handle target maneuvering changes and environmental disturbances, validating the effectiveness and superiority of neural networks in robotic target tracking tasks.

Keywords—Intelligent robots; target tracking; neural networks; adaptive control; sliding mode control

I. INTRODUCTION

Target tracking in intelligent robotics is one of the core research topics in the field of robotics, with widespread applications in numerous scenarios such as service robots, autonomous driving, drone tracking, security surveillance, and human-robot interaction [1]-[2]. Its fundamental task involves using sensors mounted on the robot (such as cameras and LiDAR) to perceive the target's position and motion, and then planning the robot's movements accordingly so that it can continuously follow the target while maintaining the desired relative pose.

However, the problem of target tracking faces numerous technical challenges. First, the motion patterns of targets are often unknown and time-varying [3]-[4]; targets may suddenly accelerate, decelerate, turn, or even change direction, and this motion uncertainty makes precise tracking difficult. Second, the environment in which the robot operates is typically dynamic and unstructured, with various interfering factors such as obstacles, changes in lighting, and sensor noise. Furthermore, the robot itself is a complex nonlinear system, with strong coupling between its dynamic characteristics and motion control. Traditional target tracking methods, such as proportional-derivative-integral control and model predictive control, typically rely on precise system models or prior knowledge of the target's motion characteristics; their adaptability and robustness in complex dynamic environments often fail to meet practical requirements.

In recent years, with the rapid development of artificial intelligence technology, neural networks have demonstrated tremendous potential in the field of robot control due to their powerful nonlinear approximation capabilities and self-learning characteristics [5]. By learning from input and output data in real time, neural networks can approximate arbitrarily complex nonlinear functions without the need for prior knowledge of an exact mathematical model. This characteristic makes them particularly well-suited for addressing motion uncertainties and system nonlinearities in target tracking. Radial Basis Function Neural Networks (RBFNNs), as a type of feedforward neural network characterized by simple structure, strong approximation capabilities, and fast learning speeds, offer distinct advantages in real-time control applications [6].

Based on the above analysis, this paper proposes a neural network-based object tracking algorithm. Its main contributions include the following three aspects:

- 1). Target motion prediction using neural networks: An RBF neural network is employed to learn the target's motion patterns in real time, and historical position data is used to predict the target's future state, thereby overcoming the reliance of traditional methods on prior knowledge of the target's motion model.
- 2). Design of an Adaptive Neural Network Sliding Mode Controller: This approach combines neural networks with sliding mode control, using the neural network to approximate the uncertain terms in the robot's dynamic model, while designing an adaptive law to adjust the network weights online, thereby ensuring system stability.
- 3). Improved weight update rule: A sliding surface is introduced as the basis for adjusting the network weights, and a weight update rule is designed based on Lyapunov stability theory to ensure the convergence of the neural network's learning process and the stability of the closed-loop system.

II. SYSTEM MODELING

A. Definition of a Coordinate System

Consider a wheeled mobile robot moving in a two-dimensional plane, whose task is to track a moving target. Define the global coordinate system $\{W\} = \{O_W, X_W, Y_W\}$ as the reference frame. The robot's pose is described by the vector $q_r = [x_r, y_r, \theta_r]^T$, where (x_r, y_r) are the robot's position coordinates in the global coordinate system, and θ_r is the robot's

orientation angle (heading angle). The target's pose is denoted as $q_t = [x_t, y_t, \theta_t]^T$.

B. Robot Kinematic Model

Wheeled mobile robots typically employ either differential drive or Ackermann steering mechanisms. This paper focuses on the differential-drive robot shown in Figure 1, whose kinematic model can be described as follows:

$$\begin{bmatrix} \dot{x}_r \\ \dot{y}_r \\ \dot{\theta}_r \end{bmatrix} = \begin{bmatrix} \cos\theta_r & 0 \\ \sin\theta_r & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_r \\ \omega_r \end{bmatrix} \quad (1)$$

Where: v_r is the robot's linear velocity, and ω_r is its angular velocity; together, they form the system's control input $u = [v_r, \omega_r]^T$. This is a typical incomplete constrained system, whose direction of motion is limited by the current orientation angle.

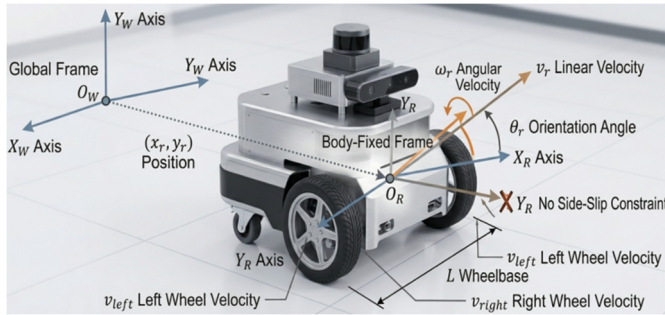


Fig. 1. Robotic Kinematic Model

C. Target Motion Model

The motion of a target is inherently uncertain and difficult to describe using precise mathematical models. Generally speaking, the motion of a target can be expressed as:

$$\dot{q}_t = f_t(q_t, w_t) \quad (2)$$

Where: f_t represents an unknown nonlinear function, and w_t represents an unknown external input (such as a control command for the target). Since an analytical form of f_t cannot be obtained, this paper employs a neural network to perform online approximation and prediction of f_t .

D. Tracking Error Model

Define the relative pose error between the robot and the target. Establish the target coordinate system at the target's center of mass, with the X_t axis pointing in the direction of the target's orientation. In this coordinate system, the relative pose error between the robot and the target is defined as:

$$\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos\theta_t & \sin\theta_t & 0 \\ -\sin\theta_t & \cos\theta_t & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x_t \\ y_r - y_t \\ \theta_r - \theta_t \end{bmatrix} \quad (3)$$

Where: e_x is the longitudinal error (along the target orientation), e_y is the lateral error (perpendicular to the target orientation), and e_θ is the orientation angle error. The goal of tracking control is to design the control input u such that the

error vector $e = [e_x, e_y, e_\theta]^T$ asymptotically converges to zero, meaning that the robot can stably follow the target and maintain the desired relative pose.

Taking the derivative of equation (3) and substituting equations (1) and (2) yields the differential equation for the tracking error:

$$\begin{bmatrix} \dot{e}_x \\ \dot{e}_y \\ \dot{e}_\theta \end{bmatrix} = \begin{bmatrix} \omega_t e_y - v_r + v_t \cos e_\theta \\ -\omega_t e_x + v_r \sin e_\theta + v_t \sin e_\theta \\ \omega_r - \omega_t \end{bmatrix} \quad (4)$$

Where: v_t and ω_t represent the target velocity and angular velocity, respectively; these are typically unknown time-varying parameters obtained through estimation or learning.

III. DESIGN OF A NEURAL NETWORK-BASED OBJECT TRACKING ALGORITHM

The target tracking algorithm proposed in this paper employs a hierarchical structure, as shown in Figure 2. The upper layer consists of a target motion prediction module, which uses a radial basis function (RBF) neural network to predict the target's future position and velocity based on its historical trajectory. The lower layer consists of an adaptive sliding mode controller, which calculates the robot's control inputs based on the predicted target state and the current tracking error. The online adjustment of the neural network weights is performed by an adaptive law driven by the sliding surface, forming a closed-loop learning mechanism.

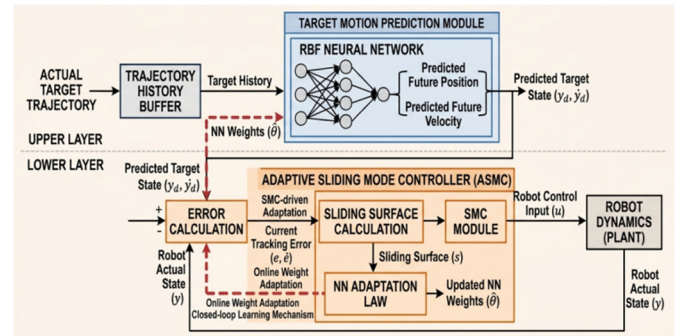


Fig. 2. Overall Structure of the Algorithm

A. Design of RBF Neural Network Architectures

An RBF neural network is a three-layer feedforward network consisting of an input layer, a hidden layer, and an output layer. The basic principle is that input vectors are mapped to a high-dimensional feature space via radial basis functions and then linearly weighted and combined in the output layer. RBF networks offer advantages such as a simple structure, high approximation accuracy, and fast learning speeds, making them particularly suitable for real-time control applications. The RBF neural network designed in this paper is used to approximate unknown nonlinear functions in the target motion model. The network architecture is as follows:

1). Input layer: The position information of the target at the n most recent time steps is used, i.e., $x_{in} = [x_t(k), x_t(k-1), \dots, x_t(k-n+1), y_t(k), \dots, y_t(k-n+1)]^T$, to predict the target's future position.

2). Hidden layer: Contains L neurons, each of which uses a Gaussian radial basis function as its activation function:

$$h_j(x_{in}) = \exp\left(-\frac{\|x_{in} - c_j\|^2}{2\sigma_j^2}\right), j = 1, 2, \dots, L \quad (5)$$

Where: c_j is the center vector of the j th basis function, and σ_j is the width parameter.

3). Output layer: Predicts the target's position at the next time step ($\hat{x}_t(k+1), \hat{y}_t(k+1)$), using a linearly weighted sum:

$$\hat{x}_t(k+1) = \sum_{j=1}^L w_{xj} h_j(\mathbf{x}_{in}), \quad \hat{y}_t(k+1) = \sum_{j=1}^L w_{yj} h_j(\mathbf{x}_{in}) \quad (6)$$

Where: w_{xj} and w_{yj} are network weights.

The approximation of an unknown nonlinear function f_t by an RBF neural network can be expressed as:

$$f_t(x_{in}) = W^{*T} h(x_{in}) + \varepsilon(x_{in}) \quad (7)$$

Where: W^* is the optimal weight matrix, $h(x_{in}) = [h_1, h_2, \dots, h_L]^T$ is the radial basis function vector, and ε is the network approximation error.

B. Design of Adaptive Learning Rules for Neural Network Weights

To enable online adjustment of neural network weights, this paper proposes an adaptive update rule based on a sliding surface. The prediction error of the neural network is defined as:

$$\tilde{x}_t(k) = x_t(k) - \hat{x}_t(k), \quad \tilde{y}_t(k) = y_t(k) - \hat{y}_t(k) \quad (8)$$

The objective of adjusting network weights is to minimize prediction error. However, due to the uncertainty inherent in the target's motion, simply minimizing prediction error does not guarantee the stability of the tracking system. Therefore, this paper combines the learning of network weights with a robot tracking control law, using tracking error to drive the update of the weights.

Define the sliding surface for robot tracking:

$$s = e_x + \lambda e_y + \mu \tanh(e_\theta) \quad (9)$$

Where: $\lambda > 0$ and $\mu > 0$ are design parameters, and $\tanh(\cdot)$ is the hyperbolic tangent function, which is used to smooth out orientation errors.

Based on Lyapunov stability theory, the adaptive update rule for neural network weights is designed as follows:

$$\hat{w}_{xj} = \eta_x \cdot s \cdot h_j(\mathbf{x}_{in}) - \kappa_x |s| \hat{w}_{xj} \quad (10)$$

$$\hat{w}_{yj} = \eta_y \cdot s \cdot h_j(\mathbf{x}_{in}) - \kappa_y |s| \hat{w}_{yj} \quad (11)$$

Where: $\eta_x, \eta_y > 0$ are the learning rates, $\kappa_x, \kappa_y > 0$ are the adjustment coefficients (used to prevent the weights from diverging), and $\hat{w}_{xj}, \hat{w}_{yj}$ are the estimated values of the weights.

C. Design of an Adaptive Slip-Mode Controller

Based on the error dynamics model shown in Equation (4), the robot's control inputs are designed. By substituting the unknown terms in Equation (4) (i.e., the target velocity and angular velocity) with values predicted by a neural network, we obtain:

$$\begin{bmatrix} \dot{e}_x \\ \dot{e}_y \\ \dot{e}_\theta \end{bmatrix} = \begin{bmatrix} \hat{w}_t e_y - v_r + \hat{v}_t \cos e_\theta \\ -\hat{w}_t e_x + v_r \sin e_\theta + \hat{v}_t \sin e_\theta \\ \omega_r - \hat{\omega}_t \end{bmatrix} + \mathbf{d} \quad (12)$$

Where: $\hat{v}_t$ and $\hat{\omega}_t$ are the target velocity and angular velocity predicted by the neural network, and $\mathbf{d} = [d_1, d_2, d_3]^T$ is the aggregate disturbance consisting of the neural network's approximation error and external disturbances, which is assumed to be bounded.

Taking the derivative of equation (9) and substituting it into equation (12) yields:

$$\dot{s} = \dot{e}_x + \lambda \dot{e}_y + \mu(1 - \tanh^2 e_\theta) \dot{e}_\theta \quad (13)$$

The design control principle is:

$$v_r = \hat{v}_t \cos e_\theta + \hat{w}_t e_y + \lambda(\hat{v}_t \sin e_\theta - \hat{w}_t e_x) + \mu(1 - \tanh^2 e_\theta)(-\hat{w}_t) + k_v s \quad (14)$$

$$\omega_r = \hat{\omega}_t - \frac{1}{\mu(1 - \tanh^2 e_\theta)} (k_\omega s + \eta \operatorname{sgn}(s)) \quad (15)$$

Where: $k_v, k_\omega > 0$ are controller gains, and $\eta > 0$ is the switching gain, used to counteract the effects of the lumped disturbance $\mathbf{d}$. The sign function $\operatorname{sgn}(s)$ is introduced to ensure the system's robustness against disturbances and to reduce oscillations.

D. Stability Analysis

Theorem 1: For a robotic system described by Equation (1) and a target system described by Equation (2), under the action of the control laws in Equations (14)-(15) and the neural network weight update laws in Equations (10)-(11), if the controller gain satisfies $k_v, k_\omega > 0$ and the switching gain $\eta > \|\mathbf{d}\|$, then the sliding surface s of the closed-loop system converges to a neighborhood of zero within a finite time, and the tracking error is bounded.

Proof of Theorem 1: Define the Lyapunov function:

$$V = \frac{1}{2} s^2 + \frac{1}{2\eta_x} \sum_{j=1}^L \tilde{w}_{xj}^2 + \frac{1}{2\eta_y} \sum_{j=1}^L \tilde{w}_{yj}^2 \quad (16)$$

Where: $\tilde{w}_{xj} = w_{xj}^* - \hat{w}_{xj}$ is the weight estimation error. Taking the derivative with respect to V , substituting the control law and the weight update law, and applying appropriate scaling, we obtain: $\dot{V} \leq -k_v s^2 - (\eta - \|\mathbf{d}\|)|s| + \text{a bounded term}$. By choosing sufficiently large values of η and k_v , we can ensure that $\dot{V} \leq 0$ holds when the sliding surface is sufficiently large, thereby proving the reachability of the sliding surface and the stability of the system. Q.E.D.

In summary, the algorithm can be summarized as shown in Figure 3:

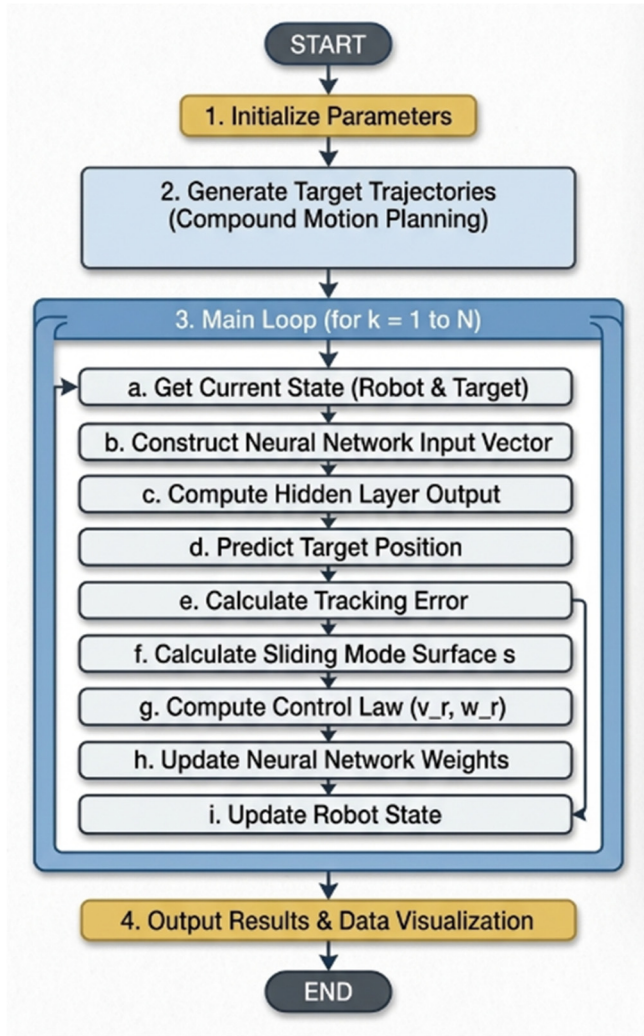


Fig. 3. Algorithm Flowchart

IV. SIMULATION EXPERIMENT

To verify the effectiveness of the proposed algorithm, a simulation platform was set up in MATLAB. The parameters are set as follows: the robot's initial position is $[0,0]^T$, initial orientation is 0 rad, maximum linear velocity is 1.0 m/s, and maximum angular velocity is 2.0 rad/s. Neural network parameters: input dimension $n = 4$ (number of historical position points), number of neurons in the hidden layer $L = 20$, basis function width $\sigma_j = 1.5$, learning rates $\eta_x = \eta_y = 0.5$, and correction coefficients $\kappa_x = \kappa_y = 0.01$. Controller parameters: $\lambda = 2, \mu = 0.5, k_v = 3, k_\omega = 2, \eta = 0.8$. Simulation duration $T = 30$ seconds, sampling time $dt = 0.02$ seconds.

Figure 4 shows the tracking trajectories of the robot using the NN-SMC algorithm, the PID algorithm, and the MPC algorithm, respectively. The target trajectory is a composite curve consisting of periodic motion and random disturbances. It can be seen that the tracking trajectory of the NN-SMC algorithm closely matches the target trajectory (black solid line), maintaining tight tracking even when the target changes direction or speed. While PID and MPC control perform reasonably well on straight sections, they exhibit significant lag

and deviation in high-curvature regions, indicating insufficient adaptability to changes in the target's motion.

Figure 5 shows the position tracking error and heading angle error. After an initial brief adjustment period, the position error of the NN-SMC rapidly converged to a low level, with an average error of approximately 0.087 m; in contrast, the average error of the PID control was approximately 0.218 m, with a noticeable error peak during the maneuvering phase; the average error of the MPC control was 0.162 m. This confirms the effectiveness of neural network online learning in compensating for target motion uncertainty.

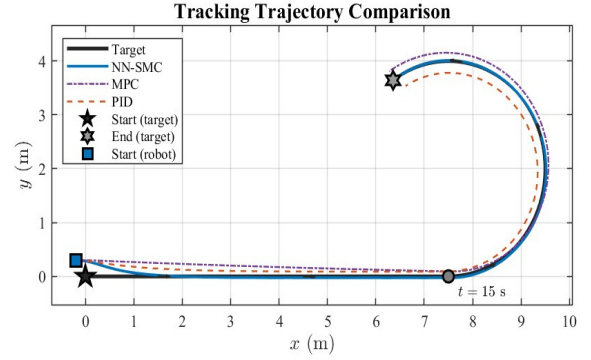


Fig. 4. Track Comparison Chart

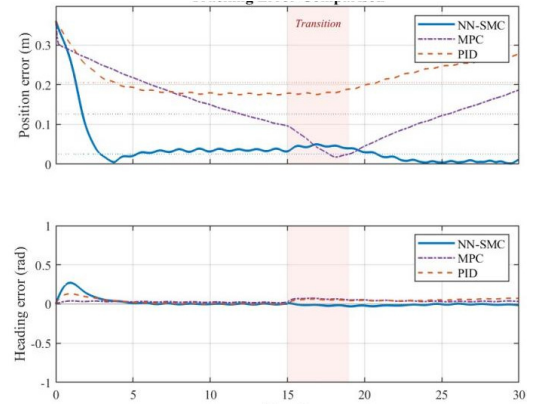


Fig. 5. Tracking Error Comparison Chart

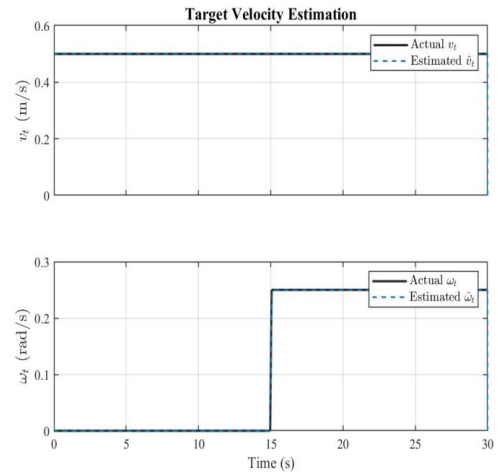


Fig. 6. Network Prediction Visualization

Figure 6 shows a comparison of the neural network’s predicted values; the two curves almost completely overlap, demonstrating the effectiveness of the RBF neural network.

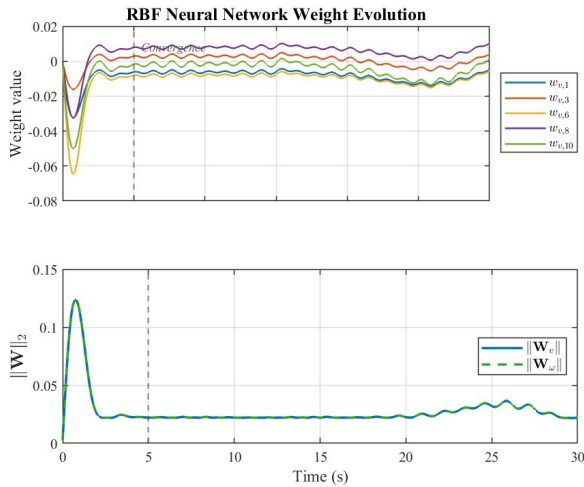


Fig. 7. Evolution of Neural Network Weights

Figure 7 shows the time-dependent trajectories of the weights of five representative neurons in the hidden layer. It can be observed that the weights adjust rapidly between 0 and 5 seconds (the convergence time is marked by a black dashed line), after which they tend to stabilize, indicating that the neural network has converged. The subfigures below show the changes in the norm of the weight vectors in the x and y directions, both of which reach a stable state after approximately 5 seconds. The damping term (κ term) in the weight update rule effectively prevents weight divergence, ensuring the stability of the online learning process.

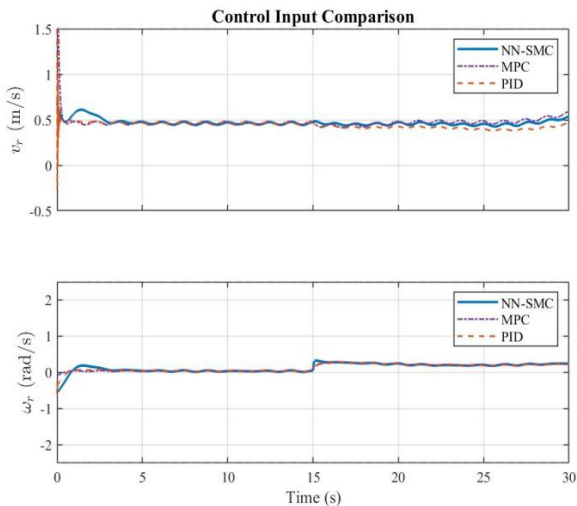


Fig. 8. Comparison Chart of Control Inputs

Figure 8 shows the linear velocity control input and angular velocity control input. The control signals from the NN-SMC are noticeably smoother than those from the PID and MPC controllers, which exhibit frequent high-frequency oscillations, particularly in the angular velocity channel. Smoother control

inputs help reduce wear and energy consumption in the actuator (motor) and improve the system’s reliability in practical applications.

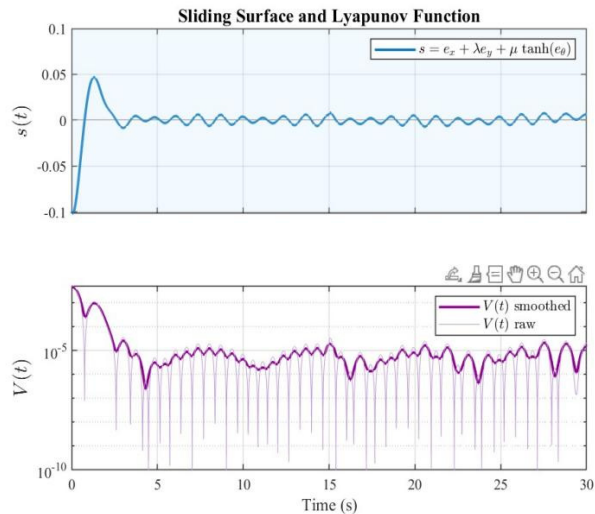


Fig. 9. Graph of the Sliding Surface and the Lyapunov Function

Figure 9 shows the variation of the sliding surface $s(t)$ over time. The sliding surface rapidly converges from an initial non-zero value to a point near zero and fluctuates slightly around zero, verifying that the sliding surface accessibility condition is satisfied. Additionally, the Lyapunov function $V(t)$ exhibits a monotonically decreasing trend and eventually converges, providing theoretical proof of the closed-loop system’s stability, consistent with the analytical conclusions of Theorem 1.

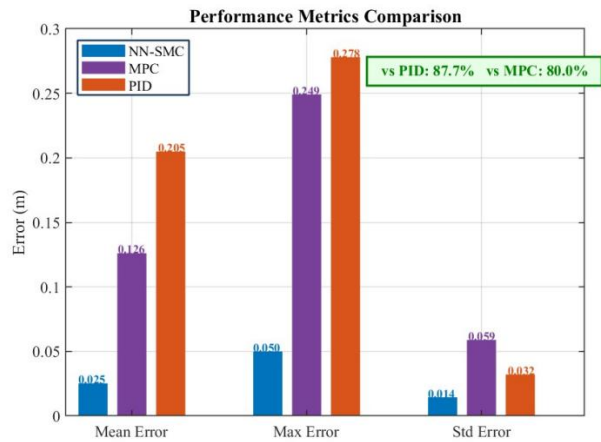


Fig. 10. Summary Bar Chart of Performance Metrics

Figure 10 illustrates the performance of these three methods across three metrics: mean squared error, maximum error, and standard deviation. NN-SMC outperforms both PID control and MPC control on all three metrics, indicating that NN-SMC not only achieves higher tracking accuracy but also demonstrates more stable performance.

V. CONCLUSION

This paper proposes a neural network-based target tracking algorithm for intelligent robots operating in complex dynamic

environments. The effectiveness of the algorithm was verified through MATLAB simulations, providing a reference for research on trajectory tracking for intelligent robots.

REFERENCES

- [1] Wang T M, Tao Y, Liu H. Current researches and future development trend of intelligent robot: A review[J]. International Journal of Automation and Computing, 2018, 15(5): 525-546.
- [2] Wei H H, Zhang Y, Sun X, et al. Intelligent robots and human-robot collaboration in the construction industry: A review[J]. Journal of Intelligent Construction, 2023, 1(1): 1-12.
- [3] Yan J, Jiao H, Pu W, et al. Radar sensor network resource allocation for fused target tracking: A brief review[J]. Information Fusion, 2022, 86: 104-115.
- [4] Zhang P, Yan J, Pu W, et al. Multi-dimensional resource management scheme for multiple target tracking under dynamic electromagnetic environment[J]. IEEE Transactions on Signal Processing, 2024, 72: 2377-2393.
- [5] Hassan N, Saleem A. Neural network-based adaptive controller for trajectory tracking of wheeled mobile robots[J]. IEEE Access, 2022, 10: 13582-13597..
- [6] Zhao J, Li Y, Li Y, et al. A predefined-time radial basis function (RBF) neural network tracking control method considering actuator faults for a new type of spraying robot[J]. Mechanical Sciences, 2025, 16(1): 51-60.